K2VIEW FABRIC SERVICES ON AMAZON AWS

K2view Data Product Platform Services Installation in an Air-Gapped Deployment

Version: 1.0

Date: 18 July 2025

Table of Contents

1. Introduction

This document provides comprehensive guidance for installing and configuring K2view Fabric Services in a secure, air-gapped Amazon Web Services (AWS) environment. It outlines the architecture, deployment models, prerequisites, and configuration options necessary to successfully deploy the K2view Data Product Platform using Kubernetes and Helm charts.

The content is tailored for enterprise environments with strict security, networking, and operational constraints. It supports both development and production-grade use cases and describes how to deploy either **Fabric Server** for runtime environments or **Fabric Web Studio** for development and proof-of-concept spaces.

Included in this guide are:

- Descriptions of K2view's supported deployment modes (Managed, Self-Hosted, Hybrid, Air-Gapped)
- Distinctions between Fabric Web Studio and Fabric Server deployments
- Detailed installation steps using Helm, including configuration via values.yaml
- Instructions for deploying supporting components like ingress controllers and in-cluster databases
- Integration guidance for Single Sign-On (SSO), Secret Managers, and persistent storage
- Best practices for namespace isolation, security, scaling, and high availability

Whether you're standing up Fabric in a private cloud, configuring spaces in isolated VPCs, or managing development pipelines in a secure Kubernetes cluster, this document will guide you through a reliable and repeatable installation process.

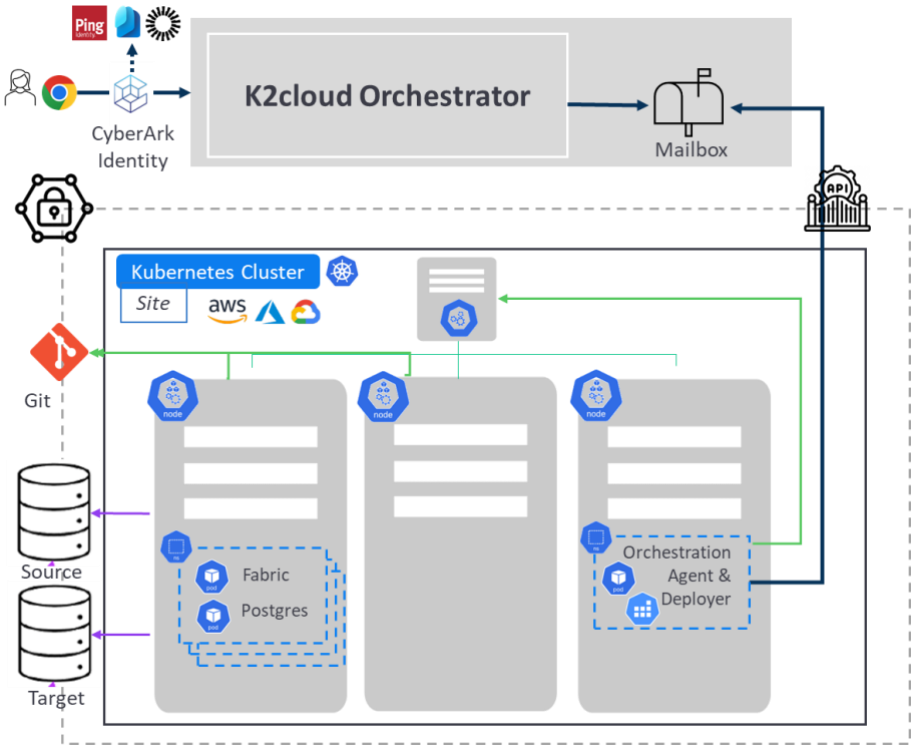
2. K2view Fabric

K2view offers different types of deployments:

- **Fully Managed**: A fully managed SaaS/PaaS deployment in cloud environments such as AWS, Azure, and Google Cloud integrated with K2cloud Orchestrator.
- **Self-hosted**: These can be on various platforms such as GKE (Google Kubernetes Engine), EKS (Amazon Elastic Kubernetes Service), AKS (Azure Kubernetes Service) across different clouds integrated with K2cloud Orchestrator.
- **Hybrid:** deployment across various cloud providers like GCP (Google Cloud Platform), AWS (Amazon Web Services), and Azure (Microsoft Azure). It represents a combination of both self-hosted and fully managed environments, utilizing services like AWS, GKE, and EKS integrated with K2cloud Orchestrator.
- **Air-Gapped:** A variation on Self-hosted or Hybrid deployment without any integration with K2cloud Orchestrator.

About K2cloud Orchestrator

The K2view Fabric Platform has two primary components: the **K2cloud Orchestration Platform** and **Kubernetes**. The **K2cloud Orchestrator** manages metadata and access control and instructs Kubernetes, while **Kubernetes** handles the actual deployment, scaling, and management of containers in a secure and efficient manner across different environments.



K2cloud Orchestration Platform Functions

Creates, controls, and monitors various Fabric runtime environments (=spaces). It performs:

- **Metadata Management**: It holds and manages metadata for projects and spaces, overseeing key information about the platform's resources.
- **Kubernetes Instructions**: It provides instructions to Kubernetes on how to build and maintain spaces, guiding Kubernetes in managing infrastructure and resources.
- **Permissions and Access Control**: The platform also sets permissions and manages access control, ensuring that the right users have secure and structured access to the right resources.

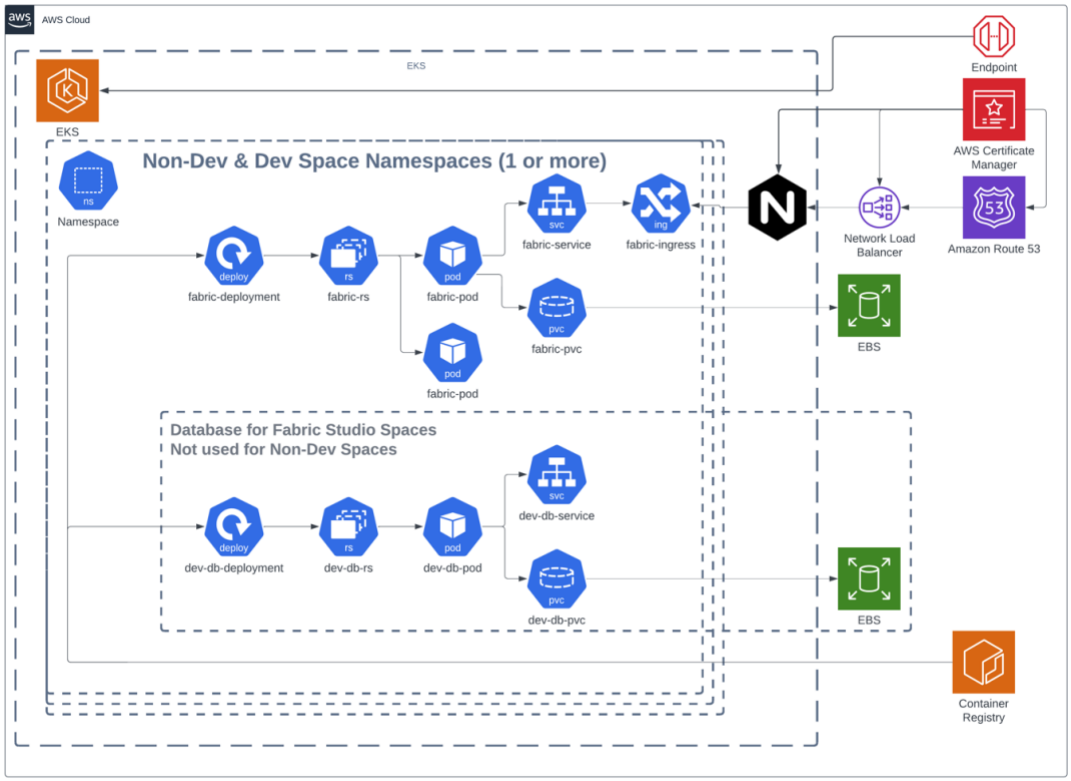
K2cloud Orchestrator leverages on-cluster services:

- **Mailbox:** Used by the K8S clusters to pull, via the K2view Fabric Agent, instructions from the K2cloud orchestration platform.
    - Each cluster has its own GUID inbox ID.
    - The Mailbox's database is encrypted.
    - The cloud platform has NO access to the K8S sites
- **K2cloud Agent:** Pulls "instructions" from K2cloud orchestration platform and to act upon.
    - Its source code can be examined/scanned (public git repo)
    - Uses a K8s service account to manage K2view Fabric spaces.
    - Holds credentials for associated services
- **K2cloud deployer:** Triggers jobs and manages the deployment of YAML specifications and in conjunction with K8s

2.1. Air-Gapped Deployment Components

This document describes an air-gapped deployment of K2view Fabric. With Air-gapped deployment - a variation on Self-hosted or Hybrid deployment - K2cloud Orchestrator, the K2cloud Agent, and K2cloud Deployer are not installed or used.

The K2view Fabric Platform is installed within **Kubernetes** in an air-gapped deployment as depicted here.



Kubernetes Functions:

The Kubernetes (K8s) Cluster is where K2view spaces are deployed. It performs:

- **Container Management**: Kubernetes is responsible for deploying and managing containers within isolated namespaces, providing a secure and organized structure for running applications.
- **Secrets, Storage, and Configuration**: Kubernetes is also responsible for managing secrets (sensitive information like passwords), storage, and configuration management, ensuring these elements are handled securely and efficiently.

K2view Components:

- **Fabric / Studio:** K2view Fabric is the core runtime engine, deployed as a scalable Kubernetes workload. Fabric Studio is the development environment layered on top of Fabric, enabling data integration, transformation, and orchestration. Studio deployments include additional components for developer workflows and are typically single-replica for simplicity.
- **Ingress.** Ingress is used to route external traffic into Fabric services. The deployment supports NGINX by default but can be adapted to ALB, AGIC, or GCE ingress controllers. Ingress routing can be configured using subdomains or paths per namespace (recommended), enabling multi-user access with centralized control and TLS termination
- **In-cluster Postgres for Studio**. Studio spaces include an internal PostgreSQL database deployed using the generic-db Helm chart. This lightweight database is used for internal metadata and Studio functions. The chart supports storage configuration, resource constraints, affinity settings, and secrets management, ensuring a secure and flexible setup for development use.

2.2. Fabric and Fabric Web Studio

The **Fabric Web Studio** and the **Fabric Server Deployment** are distinct in purpose, usage, and supporting components, even though both are built on the same Fabric runtime. Here's a breakdown of the key differences:

2.2.1. Fabric Web Studio Deployment (Dev / Studio Spaces)

Purpose:

Provides a self-contained, development-friendly environment for building and testing Fabric projects. A Fabric Web Studio deployment is required for Proof-of-Concept deployments.

Characteristics:

- **Includes Web Studio UI**: Offers a web-based interface for developers to design flows, define LUs, and manage configurations.
- **Single-replica Deployment**: Typically deployed as a single pod (Deployment) for simplicity.
- **Embedded Database**: Ships with an internal PostgreSQL DB via the generic-db Helm chart for metadata storage (e.g., Studio repository, project assets).
- **Not production-grade**: Intended for development and testing only; not horizontally scaled or HA-configured.
- **Simplified storage**: Uses a PVC for workspace persistence; supports faster spin-up and teardown.

2.2.2. Fabric Server Deployment (Runtime / Non-Dev Spaces)

Purpose:

Runs the Fabric engine in production, staging, QA, or other non-development environments.

Characteristics:

- **Stateless or Stateful**: Typically deployed as a StatefulSet with multiple replicas for high availability.
- **No embedded DB**: Assumes external databases (e.g., customer-managed Postgres, Oracle, etc.) for System DB and data sources/targets.
- **Optimized for scale**: Configured for performance, autoscaling, and horizontal distribution.
- **Ingress-enabled**: Exposed through ingress controllers for secure and controlled external access.
- **No Studio UI**: No web-based development interface included—used strictly for runtime execution of jobs, APIs, etc.

2.2.3. Summary Table

Feature	Fabric Web Studio	Fabric Server
Primary Use	Development / Studio use	Production / Non-Dev runtime
Deployment Type	Deployment	StatefulSet
Replicas	1	1 or more
UI Interface	Includes Studio UI	No Studio UI
Internal Database	Internal Postgres via Helm	External DB required
Scaling	Not intended for scale	Horizontal & autoscaling supported
Persistence	Required PVC for workspace persistence	Optional; external storage preferred
Environment Target	Local Dev, Feature Branch Spaces	QA, UAT, Production

2.3. Installation using Fabric Helm Chart

2.3.1. The Fabric Helm Chart

The Fabric Helm Chart is described in the README.md section and distributed via https://github.com/k2view/blueprints. It is used to deploy Fabric Web Studio – for development use - and Fabric for non-development (e.g., integration testing, pre-production and production) deployments.

The Fabric Helm chart is designed to deploy the Fabric application on Kubernetes. This chart includes a variety of configurable options for environment variables, resource allocation, storage, ingress, and network policies to ensure a robust, secure, and high-performance deployment.

The values.yaml file is used to provide settings be needed. Using the Fabric Helm Chart, a K2view Space is created by running a single Helm command:

```
helm install -f values.yaml [space-name]
```

A namespace "space-name" will be created in the Kubernetes cluster along with all required components to create a Fabric Space.

Once the Space is up and running (pod in READY state), you can access it via browser by navigating to the URL defined in {{ .Values.fabric.ingress.host }}/space-name. E.g.,

```
https://dev-cluster.mycompany.com/space-1
```

2.3.2. Other Helm Charts

The Fabric Helm Chart's manifest is detailed in the README.md section of this document. The other relevant charts are:

- **Generic DB**: https://github.com/k2view/blueprints/tree/main/helm/generic-db. The generic-db Helm chart is designed to deploy a database like PostgreSQL on Kubernetes clusters. It supports configuring resource requests and limits, storage options, affinity rules, and secrets management, making it flexible for different environments and use cases.
- **Ingress**: https://github.com/k2view/blueprints/tree/main/helm/ingress-nginx-k2v. This Helm chart is designed to deploy a Nginx ingress controller in a Kubernetes environment. This controller manages the routing of HTTP and HTTPS traffic to the appropriate services within the cluster. It is particularly configured for the K2view site, ensuring optimized performance and security.
- **Kubernetes resource templates** that define how each service is deployed, scaled, networked, and stored.

2.4. Configuring Single Sign-On (SSO)

K2view Fabric supports SAML-based SSO.

- Refer to the Appendix – SSO / IDP Reference section for an overview of the necessary configuration.
- Information required will be a base64-encoded private key and the provider's SSO parameters.
- To learn about how to configure SSO and its capabilities, please refer to https://support.k2view.com/Academy/articles/26_fabric_security/13_user_IAM_configuration.html#saml-configuration

2.5. Secret Manager Integration

K2view's Fabric platform integrates seamlessly with popular Secret Manager services to protect sensitive information such as passwords. These integrations ensure that secrets used in interfaces for communication with external systems are not stored within Fabric itself, enhancing security. The supported secret management providers include:

- AWS Secret Manager
- HashiCorp Vault
- Azure Key Vault
- CyberArk CCP
- Google Secret manager
- OneIdentity Safeguard

For more information:
https://support.k2view.com/Academy/articles/26_fabric_security/04a_secret_manager.html

3. Prerequisites

To install K2view Fabric Services, populate your container registry (e.g., Amazon ECR), and create a K2view Project and a Space, you must meet specific prerequisites and take planning steps.

3.1. Internet Access

Internet access is required **by the installation team** for:

1. Github.com to clone K2view's blueprints at `https://github.com/k2view/blueprints.git`
2. K2view's Nexus Docker Image repository at `https://docker.share.cloud.k2view.com`
    a. These images are intended to be pulled from K2view and pushed into the customer's image container registry
3. If you plan to install TDM, you will need to download components from K2view's Exchange

No access to the internet is required otherwise or at all for the Kubernetes deployment.

3.2. DNS

Using Context-Based URLs for your spaces allows you to define a single DNS A record. This is available with Fabric 8.2 and later.

If you use a preexisting DNS zone, you need this domain name to create your Ingress Controller and register a DNS A record. You must create a DNS record pointing to the node hosting the K8s Ingress Controller (it can point to a private or a public IP). The DNS record for the server hosting the K8s node does not need to be registered in the public DNS. The only requirement is that users can resolve the DNS name internally to the customer's environment.

DNS Configuration

Please use AWS Route53 to create and manage DNS records and create the following DNS record:

Type: A (or CNAME depending on your DNS setup)

3.3. TLS keys and certificates

When using K2view's AWS Blueprint and Terraform, a TLS PEM certificate and its associated TLS PEM Key file are needed. The ingress controller will use this certificate to terminate TLS connections.

Your network team needs to create a TLS certificate for the domain you chose as your Domain Ingress. The complete certificate authority chain must be present. During installation, you must indicate the path to your PEM TLS certificate and its corresponding private key.

3.4. Tools

The following tools are required to perform the installation by the user with the required Permissions and Roles.

- Terraform – version 1.9.x - HashiCorp install guide
- Kubectl – version 1.14.0 - Kubectl install guide
- AWS CLI – latest version - Installing the AWS CLI
- Helm – version 2.13.0 - Helm install guide
- Docker – latest version - You can use the tools provided by your container registry to pull from the K2view Nexus and push to your registry. Docker or an OCI-compatible tool will likely be needed for this.

3.5. Git code repository

A Git code repository is required for your Fabric Project. Three pieces of information will be required to create your K2view project including:

- Repository URL
- Git token
- Git Branch / Tag

3.6. Container Registry

K2view will create a Nexus account, user accounts, and associated configurations for you. You will use the K2view Nexus account to obtain K2view Docker images.

You must populate your container registry by pulling and pushing docker images. Docker must be installed on a computer to pull and push docker images from the K2view Nexus repository to the container created. You will need a K2view Nexus account you can obtain from your K2view representative.

Extracting and pushing Docker images to your Container Registry

- You will need the K2view Nexus account to perform the “pull” action. This will be provided to you by your K2view representative.
- You will need access to your container registry to publish K2view and other Docker images.

3.7. Persistent Storage Selection

When designing storage solutions for applications running in AWS, selecting the right type of persistent storage is important to ensuring availability, reliability, and scalability. AWS offers multiple options for persistent storage, including Amazon Elastic Block Store (EBS) and Amazon Elastic File System (EFS). Each solution is tailored to specific use cases and operational needs, and understanding their pros and cons is essential for optimal implementation. The following subsections explore the limitations and benefits of EBS and EFS, guiding when to choose one over the other.

EBS and Its Limitations

Amazon Elastic Block Store (EBS) provides high-performance block storage for single availability zone (AZ) use cases, but its AZ-bound nature presents challenges in multi-AZ environments.

1. **AZ-Bound Nature**. Amazon Elastic Block Store (EBS) volumes are tied to a specific AZ. If a pod using an EBS volume is terminated and rescheduled in another AZ, it cannot access the original volume because it cannot be mounted across AZs.
2. **Persistent Pod Needs**. Your pod needs a persistent storage layer to retain data beyond its lifecycle. EBS can meet this requirement **only if the pod remains in the same AZ** where the EBS volume exists.
3. **Pod Scheduling and Multi-AZ Clusters**. In Kubernetes clusters running in EKS, pods can be rescheduled in any AZ within the cluster. If a pod moves to another AZ, the storage layer (EBS volume) becomes inaccessible, causing a failure to retrieve persistent data.

Why Consider EFS

Amazon Elastic File System (EFS) provides a flexible and scalable storage solution for workloads that demand cross-AZ accessibility, shared access, persistent storage, and seamless scalability.

1. **Cross-AZ Access**. EFS is a **managed NFS (Network File System)** accessible across all AZs within the same region. It allows any pod, regardless of its AZ, to access the shared filesystem seamlessly.
2. **Shared Access**. EFS supports multiple clients accessing the filesystem simultaneously, making it ideal for workloads where multiple pods might need to read or write from the same storage.
3. **Persistence Across Pod Lifecycle**. EFS ensures data remains persistent even if the pod is terminated and recreated in a different AZ.
4. **Scalability**. EFS automatically scales as your data grows, eliminating the need for manual resizing.

When Should You Use EFS Over EBS

You should choose EFS when:

- You are required to run in a **multi-AZ EKS cluster**.
- Your pods may be rescheduled across different AZs.

4. Security Overview

4.1. K8s Namespace-based Isolation

In the K2view Fabric platform, Kubernetes (K8S) Namespaces are crucial in organizing and isolating various runtime environments. A Namespace is an isolated group within a Kubernetes cluster, consisting of organized pods and all related services. These namespaces function like virtual clusters inside the larger Kubernetes cluster, enabling the logical separation of workloads.

Within K2view Fabric, the term “spaces” refers to these namespaces. Each space contains dedicated services and pods, ensuring that the workloads, such as Fabric and database components, are independently managed and securely isolated.

This isolation model enhances security, resource management, and operational efficiency, ensuring that each namespace can function independently without affecting the others. Using Kubernetes namespaces, K2view Fabric provides a structured and secure environment for deploying, scaling, and managing services in a distributed system.

Data Connection Authentication Methods

K2view Fabric provides multiple methods for securing data connections when connecting from Fabric to data sources, as well as when access to Fabric APIs.

For more information:
https://support.k2view.com/Academy/articles/26_fabric_security/05_fabric_webservices_security.html

Securing the Transfer Level:

K2view Fabric implements TLS/HTTPS protocols to secure data transmission and ensure encrypted communication between systems. Additionally, external data providers, whether the source or the target, can be connected through VPC peering or a VPN, with further security enforced by whitelisting the Fabric IP or host.

4.2. Data Encryption

K2view Fabric offers robust data encryption and masking features to ensure the highest levels of security for sensitive information. These features help protect data at rest and in transit, safeguarding against unauthorized access or breaches.

Built-in Key Management Service (KMS):

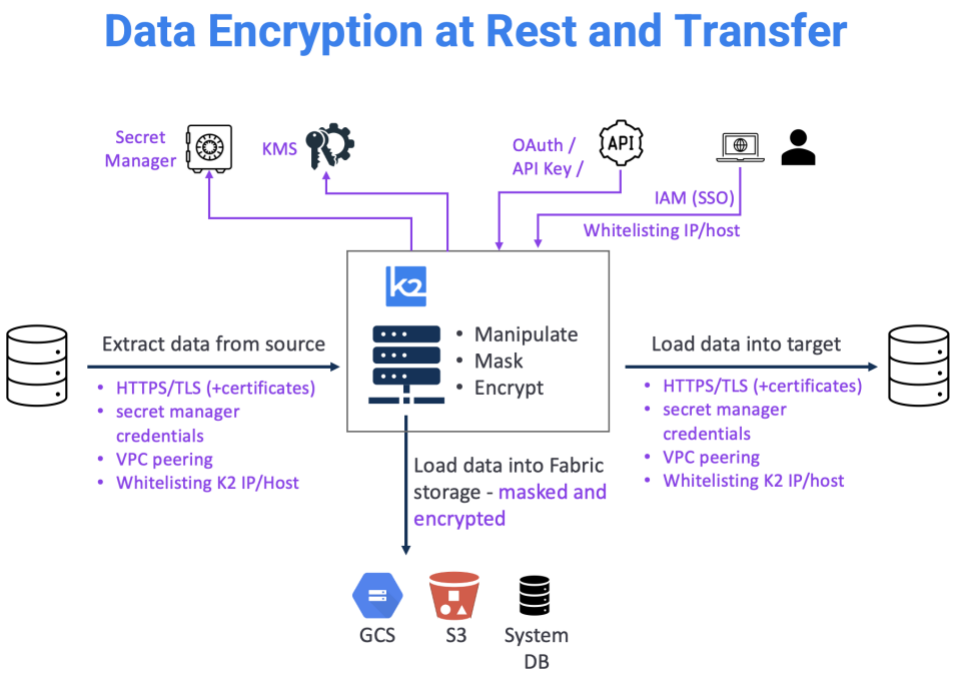
K2view Fabric provides a built-in KMS that generates a strong master encryption key in compliance with FIPS 140-2 standards. This key is securely generated and stored, ensuring that all sensitive data is protected according to stringent security regulations.

External KMS Integration:

The platform also supports integration with external KMS providers such as AWS, Google Cloud Platform (GCP), and KMIP, offering customers the flexibility to use their preferred encryption management systems.

Strong Encryption Algorithm:

K2view Fabric uses the industry-standard AES-256 encryption algorithm for data protection. This algorithm is paired with an Initialization Vector (IV), further enhancing the security and randomness of the encryption process.



Unique Encryption Methodology:

Each **Logical Unit of Information (LUI)** or data-product instance within K2view Fabric is encrypted individually and stored separately, ensuring that data remains isolated and secure from other instances.

For more information:

- **Encryption and KMS**: https://support.k2view.com/Academy/articles/26_fabric_security/02_fabric_entities_design.html
- **FIPS Compliance**: https://support.k2view.com/Academy/articles/26_fabric_security/18_FIPS_implementation.html

4.3. Data Protection at Rest

Storage Data Encryption in K2view Fabric

K2view Fabric offers a highly secure encryption model for storing data, particularly for Logical Unit Instances (LUIs) within micro-databases. This approach ensures that each instance is encrypted and stored separately, providing enhanced protection against unauthorized access or breaches.

Micro-Database Encryption:

Each **micro-database** that hosts a data product instance (LUI) is individually encrypted. This ensures that if any instance were compromised, the rest of the data remains safe, as only the data from the compromised instance would be affected.

Logical Unit Compression and Encryption:

The **Logical Units (LU)** are compressed and encrypted within these micro-databases, which results in incredible performance and enhanced security. The combination of compression and encryption allows K2view Fabric to handle large datasets efficiently while maintaining robust protection.

5. Installation in an Air-Gapped AWS Environment

5.1. Overview

The Fabric Helm chart provides a robust, production-ready deployment of the Fabric application on Kubernetes clusters. This chart is designed for flexibility, security, and ease of use, supporting a wide range of configuration options to suit enterprise and cloud-native environments. It is suitable for both development and production deployments and is maintained with best practices for reliability and scalability.

Features

- **Configurable Deployments:** Easily customize replicas, resources, and environment variables.
- **Production-Ready Defaults:** Secure and scalable out-of-the-box settings.
- **Support for Ingress:** Integrate with popular ingress controllers for external access, including:
  - **NGINX Ingress Controller** (default, fully supported)
  - **AWS ALB Ingress Controller** (annotation changes may be required)

Note: Some ingress controllers may require minor changes to annotations or configuration. Refer to the documentation of your chosen ingress controller for details.

- **Persistent Storage:** Optional persistent volume claims for data durability.
- **Customizable Service Exposure:** Choose between ClusterIP, NodePort, or LoadBalancer.
- **Health Checks:** Liveness and readiness probes for robust operation.
- **Resource Management:** Fine-grained control over CPU and memory requests/limits.
- **Secrets Management:** Integrate with Kubernetes secrets for sensitive data.
- **Extensible:** Easily override or extend with your own values files.

5.2. Prerequisites

- Kubernetes 1.27+
- Helm 3.0+
- Ingress controller (e.g., NGINX)

5.3. Installation

5.3.1. Add the Helm Repository (Remote Installation)

```
helm repo add fabric https://nexus.share.cloud.k2view.com/repository/fabric/
helm repo update
```

5.3.2. Install the Chart from Remote Repository

```
helm install SPACE_NAME fabric/fabric \
  --namespace SPACE_NAME \
  --create-namespace
```

5.3.3. Install the Chart Locally (After Cloning the Repo)

If you have cloned this repository, you can install the chart directly from the local directory:

```
cd helm/charts/fabric
helm install SPACE_NAME . \
  --namespace SPACE_NAME \
  --create-namespace
```

5.3.4. Custom Installation

You can override default values using a custom values.yaml file:

```
# For remote installation
helm install SPACE_NAME fabric/fabric -f my-values.yaml
# For local installation
helm install SPACE_NAME . -f my-values.yaml
```

5.4. Upgrading

To upgrade an existing release:

```
helm upgrade SPACE_NAME fabric/fabric -f my-values.yaml
```

5.5. Uninstallation

To uninstall the release:

```
helm uninstall SPACE_NAME
```

5.6. Configuration

The following table lists the main configurable parameters of the Fabric chart and their default values. For a full list and detailed descriptions, see values.yaml in the chart directory.

Parameter	Type	Default	Description
global.labels	object	[]	Additional global labels to add to resources
namespace.create	bool	false	Whether to create the namespace
namespace.name	string	"space-tenant"	Namespace to deploy into
deploy.type	string	Deployment	Deployment type (Deployment or StatefulSet)
serviceAccount.create	bool	true	Create a new service account
serviceAccount.name	string	""	Name of the service account (empty for new)
serviceAccount.provider	string	""	Cloud provider for service account
serviceAccount.arn	string	""	AWS IAM role ARN
serviceAccount.project_id	string	""	GCP project ID
serviceAccount.cluster_name	string	""	Cluster name

Parameter	Type	Default	Description
serviceAccount.azure_client_id	string	""	Azure Managed Identity client ID
container.replicas	int	1	Number of Fabric pods
container.annotationsList	list	[{{name: description, value: Fabric on Kubernetes}}]	List of pod annotations
container.resource_allocation.limits.memory	string	8Gi	Memory limit
container.resource_allocation.limits.cpu	string	2	CPU limit
container.resource_allocation.requests.memory	string	2Gi	Memory request
container.resource_allocation.requests.cpu	string	0.4	CPU request
container.image.url	string	""	Fabric image URL
container.image.repoSecret.name	string	registry-secret	Image pull secret name
container.image.repoSecret.enabled	bool	false	Enable image pull secret
storage.pvc.enabled	bool	true	Enable persistent volume claim
storage.securityContext	bool	true	Enable pod security context
storage.class	string	managed	Storage class name
storage.alocated_amount	string	10Gi	PVC size
scaling.enabled	bool	false	Enable autoscaling
scaling.minReplicas	int	1	Minimum replicas for autoscaling
scaling.maxReplicas	int	1	Maximum replicas for autoscaling
scaling.targetCPU	int	90	Target CPU utilization for HPA
networkPolicy.egress.enabled	bool	false	Enable egress network policy
networkPolicy.ingress.enabled	bool	false	Enable ingress network policy
ingress.enabled	bool	true	Enable ingress
ingress.class_name	string	nginx	Ingress class name

Parameter	Type	Default	Description
ingress.type	string	nginx	Ingress type
ingress.host	string	space-tenant.domain	Ingress host
ingress.path	bool/string	false	Boolean true uses same value as namespace or hardcoded "string"
ingress.subdomain	bool/string	false	Boolean true uses same value as namespace or hardcoded "string"
ingress.appgw_ssl_certificate_name	string	""	Azure Application Gateway SSL certificate name
ingress.tlsSecret.enabled	bool	false	Enable TLS secret for ingress
ingress.cert_manager.enabled	bool	false	Enable cert-manager integration
ingress.cert_manager.cluster_issuer	string	""	cert-manager ClusterIssuer name
ingress.custom_annotations.enabled	bool	false	Enable custom ingress annotations
ingress.custom_annotations.annotations	list	See values.yaml	Custom ingress annotations
mountSecret.enabled	bool	false	Mount decoded secret to pod
mountSecret.name	string	config-secrets	Name of the secret to mount
mountSecret.mountPath	string	/opt/apps/fabric/config-secrets	Path to mount the secret
mountSecret.data	object	See values.yaml	Data to mount as secret (see config, cp_files, idp_cert)
mountSecretB64enc.enabled	bool	false	Mount base64-encoded secret to pod
mountSecretB64enc.name	string	mount-b64-secrets	Name of the base64 secret to mount
mountSecretB64enc.mountPath	string	/opt/apps/fabric/config-secrets	Path to mount the base64 secret

Parameter	Type	Default	Description
mountSecretB64enc.data	object	See values.yaml	Data to mount as base64 secret
secretsList	list	[]	List of additional secrets (see structure in values.yaml)
initSecretsList	list	[]	List of init container secrets (see structure in values.yaml)
affinity.type	string	none	Pod affinity/anti-affinity/none
affinity.label.name	string	topology.kubernetes.io/zone	Affinity label name
affinity.label.value	string	region-a	Affinity label value

Tip: For advanced configuration and secret formats, refer to the comments in values.yaml.

5.7. Deployment Options

The Fabric Helm Chart supports two deployment types, each suited for different environments and use cases:

5.7.1. Deployment (Recommended for Development/Studio)

Use this mode for development environments or when running Fabric Studio. This mode is stateless and works **only** in a **single-replica** setup.

Recommended configuration:

- `deploy.type: Deployment`
- `container.replicas: 1`
- `storage.pvc.enabled: true`

5.7.2. StatefulSet (Recommended for Production/Server)

Use this mode for staging or production environments, or when running Fabric Server. This mode is designed for stateful workloads and high availability.

Recommended configuration:

- `deploy.type: StatefulSet`
- `storage.pvc.enabled: false (shared PVC is not recommended for multiple replicas)`
- `Set container.replicas to the desired number of server instances`

Note:

- For most production scenarios, use StatefulSet with persistent storage managed outside the pod (e.g., cloud-native databases, object storage).
- Only use a shared PVC (storage.pvc.enabled: true) if your storage class supports multi-attach and your use case requires it.

5.8. Fabric Application Configuration

The Fabric application is configured via the config.ini file, which is managed through the mountSecret.data.config variable in your values.yaml.

To customize the application configuration:

- Edit the mountSecret.data.config field in your values.yaml file.
- Provide your configuration in standard INI format, with sections and key-value pairs.

Example:

```
[fabric]
WEB_SESSION_EXPIRATION_TIME_OUT=540
ENABLE_BROADWAY_DEBUG_SERVLET=true

[system_db]
SYSTEM_DB_TYPE=SQLITE
SYSTEM_DB_HOST=/opt/apps/fabric/workspace/internal_db

# Add additional sections and keys as needed
```

Tip: See the default values.yaml for more configuration examples and comments. Changes to this section will be reflected in the Fabric application's runtime configuration under /opt/apps/fabric/workspace/config/config.ini.

5.9. RBAC

RBAC (Role-Based Access Control) in this Helm chart is managed via the serviceAccount section in your values.yaml file.

- If serviceAccount.create: true, a dedicated Kubernetes ServiceAccount will be created in the Fabric namespace and automatically associated with the Fabric pods.
- If serviceAccount.create: false, you must specify the name of an existing ServiceAccount to use.

This mechanism is especially useful in cloud environments to enable IAM-based access to cloud resources (such as AWS IAM Roles for Service Accounts, GCP Workload Identity, or Azure Managed Identity).

To enable cloud provider integration for IAM-based access, set the serviceAccount.provider field to one of the supported values:

Provider	Value for serviceAccount.provider	Additional Required Fields
AWS	aws	arn (IAM Role ARN)
GCP	gcp	gcp_service_account_name or project_id + cluster_name
Azure	azure	azure_client_id

Refer to the comments in values.yaml for detailed configuration examples for each cloud provider.

5.10. Horizontal Pod Autoscaling (HPA)

Horizontal Pod Autoscaling (HPA) is managed via the scaling section in your values.yaml file.

To enable HPA:

- Set scaling.enabled: true
- Adjust scaling.minReplicas and scaling.maxReplicas to define the allowed range of pod replicas
- Set scaling.targetCPU to the target average CPU utilization percentage that will trigger scaling

Example:

```
scaling:
  enabled: true
  minReplicas: 2
  maxReplicas: 5
  targetCPU: 80
```

Note:

- HPA requires resource requests and limits to be set for CPU in your container configuration.
- The chart will automatically create the necessary Kubernetes HPA resource when enabled.

5.11. Ingress

To enable ingress, set ingress.enabled: true and configure the hostname:

```
ingress:
  enabled: true
  class_name: "nginx"
  type: "nginx"
  host: fabric.example.com
  tlsSecret:
    enabled: false
  cert_manager:
    enabled: false
  custom_annotations:
    enabled: false
    annotations:
      - key: nginx.ingress.kubernetes.io/proxy-body-size
        value: "0"
```

5.11.1. Ingress Routing Types

The Fabric Helm chart supports two primary ingress routing strategies, allowing flexibility based on your domain and certificate setup. The routing logic is controlled by the ingress.path and ingress.subdomain parameters:

- ingress.path: If set to true, the path will be set to the namespace name. If set to a string, that string will be used as the path. If set to false, path-based routing is disabled.
- ingress.subdomain: If set to true, the subdomain will be set to the namespace name. If set to a string, that string will be used as the subdomain. If set to false, subdomain-based routing is disabled.

This allows for flexible ingress host and path generation, supporting both domain-based and path-based routing, as well as custom values for advanced scenarios.

Summary:

- Use ingress.subdomain for subdomain-based routing (e.g., space-tenant.domain).
- Use ingress.path for path-based routing (e.g., domain/space-tenant).
- You can set either to true (use namespace), a string (custom value), or false (disable).

Below are the two most common routing strategies:

1. Path-Based Routing (No Wildcard TLS) (Recommended)

Use this method if you do not have a wildcard TLS certificate. Each space is accessed via a unique path on a shared domain (e.g., domain/space-tenant). This is the recommended configuration.

- **Ingress host:** Static (e.g., domain)
- **Ingress path:** Dynamic (per space)

How to use:

- Set ingress.host to your domain (e.g., domain)
- Set ingress.path to the space name (e.g., space-tenant)
- Optional: set ingress.path to true to use namespace name as a path prefix

Example values.yaml:

```
ingress:
  enabled: true
  host: domain
  path: space-tenant
  # ...other values...
```

Resulting Ingress manifest:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: fabric-ingress
  namespace: space-tenant
  labels:
    app: fabric
  annotations:
    nginx.ingress.kubernetes.io/proxy-body-size: "0"
    nginx.ingress.kubernetes.io/proxy-read-timeout: "86400"
```

```
    nginx.ingress.kubernetes.io/proxy-send-timeout: "900"
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - domain
  rules:
    - host: domain
      http:
        paths:
          - path: /space-tenant
            pathType: Prefix
            backend:
              service:
                name: fabric-service
                port:
                  number: 3213
```

Note:

- Choose the routing type that matches your certificate and DNS setup.
- The chart templates are designed to support both strategies out-of-the-box.
- For advanced ingress controller features or custom annotations, refer to your ingress controller's documentation.

2. Domain-Based Routing (Wildcard TLS)

This method is recommended when you have a wildcard TLS certificate for your domain. Each space is accessed via a subdomain (e.g., space-tenant.domain).

- **Ingress host:** Dynamic (per space, as subdomain)
- **Ingress path:** Static (/)

How to use:

- Set ingress.host to the subdomain (e.g., space-tenant.domain)
- Set ingress.path to / or leave it blank (default)
- Optional: set ingress.subdomain to true to use namespace name as subdomain

Example values.yaml:

```
ingress:
  enabled: true
  host: space-tenant.domain
  # ...other values...
```

Resulting Ingress manifest:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: fabric-ingress
  namespace: space-tenant
  labels:
    app: fabric
  annotations:
    nginx.ingress.kubernetes.io/proxy-body-size: "0"
```

```
    nginx.ingress.kubernetes.io/proxy-read-timeout: "86400"
    nginx.ingress.kubernetes.io/proxy-send-timeout: "900"
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - space-tenant.domain
  rules:
    - host: space-tenant.domain
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: fabric-service
                port:
                  number: 3213
```

5.12. Storage (Persistence)

To enable persistent storage, configure the following:

```
storage:
  pvc:
    enabled: true
  class: managed
  alocated_amount: 20Gi
  securityContext: true
```

Note: Enabling pvc.enabled: true will create a shared PersistentVolumeClaim (PVC) for the deployment. In multi-node or highly available setups, this is generally not recommended when running more than one replica, as most storage classes do not support multi-node (ReadWriteMany) access and most use cases do not require shared storage. For most deployments with replica count above 1, set pvc.enabled: false unless you have a specific need and your storage class supports multi-attach.

5.13. Environment Variables

You can set environment variables for fabric by adding them under secretsList.data in values.yaml:

```
secretsList:
  - name: common-env-secrets
    data:
      MY_ENV_VAR_1: 'VALUE_1'
      MY_ENV_VAR_2: 'VALUE_2'
```

5.14. Troubleshooting

- Check pod logs: kubectl logs <pod-name> -n fabric
- Describe resources: kubectl describe pod <pod-name> -n fabric
- Verify ingress and service endpoints.

Appendix – SSO / IDP Reference

The values.yaml is used for configuring SSO. To learn about configuring SSO and its capabilities, please refer to https://support.k2view.com/Academy/articles/26_fabric_security_iam/13_user_IAM_configuration.html - saml-configuration.

Microsoft Entra ID specific configuration is provided here: https://support.k2view.com/Academy/articles/26_fabric_security_iam/14_user_IAM_SAML_Azure_AD_setup.html.

The following two sections of the values.yaml need to be configured to enable SSO with Microsoft Entra ID.

The initSecretsList Section

```
  - name: config-init-secrets
    data:
      CONFIG_UPDATE_FILE: /opt/apps/fabric/config-secrets/config
      K2_GIT_BRANCH: ""
      K2_GIT_PATH_IN_REPO: ""
      K2_GIT_REPO: ""
      K2_GIT_TOKEN: ""
      START_FABRIC: "false"
      # IDP_CERT_PATH: /opt/apps/fabric/config-secrets/IDP_CERT
      # IDP_ROLE: <<<ENTRA_ID_GROUP_NAME>>>
```

The mountSecret Section

```
mountSecret:
    data:
      config: |
        common_area_pubsub|TYPE|MEMORY
        ….
      #   fabric|WEB_AUTHENTICATION_PROTOCOL|SAML
      #   fabric|SERVER_AUTHENTICATOR|block_all
      #   saml|SP_ENTITYID|https://[Ingress]/space-name
      #   saml|IDP_ENTITYID|https://sts.windows.net/00000000-0000-0000-0000-000000000000/
      #   saml|IDP_SINGLE_SIGN_ON_SERVICE_URL|https://login.microsoftonline.com/00000000-0000-
0000-0000-000000000000/saml2
      #   saml|IDP_SINGLE_LOGOUT_SERVICE_URL|https://login.microsoftonline.com/00000000-0000-
0000-0000-000000000000/saml2
      #   saml|SP_CERT_ALIAS|sp_cert
      #   saml|IDP_CERT_ALIAS|idp_cert
      #   saml|SECURITY_WANT_NAMEID_ENCRYPTED|false|ADD
      # IDP_CERT: |
      #   -----BEGIN CERTIFICATE-----
      #   MII.....
      #   -----END CERTIFICATE-----
```